PLANO DE DESENVOLVIMENTO
Sistema de Gestão de Estoque em Python

Divisão de tarefas para 5 integrantes • Python • Git/GitHub • Commits individuais

Este plano adapta a separação de tarefas do projeto-base para Python, mantendo cinco responsáveis, branches próprias, testes e integração controlada. O histórico de commits é tratado como parte da avaliação do trabalho.

1. Estrutura do projeto

```
controle-estoque/
├── src/
│   ├── main.py
│   ├── banco.py
│   ├── produtos.py
│   ├── categorias.py
│   ├── movimentacoes.py
│   ├── fornecedores.py
│   ├── funcionarios.py
│   ├── autenticacao.py
│   ├── relatorios.py
│   └── auditoria.py
├── tests/
│   ├── test_produtos.py
│   ├── test_categorias.py
│   ├── test_movimentacoes.py
│   ├── test_fornecedores.py
│   ├── test_funcionarios.py
│   ├── test_autenticacao.py
│   └── test_relatorios.py
├── requirements.txt
├── README.md
└── .gitignore
```

Regra de Git: cada pessoa trabalha em sua própria branch criada a partir de develop. Commits devem ser pequenos, objetivos e relacionados a uma única entrega.

2. Organização das branches

```
main
└── develop
    ├── feature/produtos
    ├── feature/movimentacoes
    ├── feature/relatorios
    ├── feature/autenticacao
    └── feature/fornecedores-integracao
```

Fluxo recomendado para todos:

```
git checkout develop
git pull origin develop
git checkout -b feature/NOME-DA-FEATURE
```

PESSOA 1 — Produtos e Categorias

Branch:

```
git checkout develop
git pull origin develop
git checkout -b feature/produtos
```

Commit 1 — Categorias

- Arquivos: src/categorias.py
- Implementar cadastrar_categoria(), listar_categorias(), buscar_categoria() e editar_categoria().

```
git add src/categorias.py
git commit -m "feat: implementa gerenciamento de categorias"
```

Commit 2 — Produtos

- Arquivos: src/produtos.py
- Implementar cadastrar_produto(), listar_produtos(), buscar_produto(), editar_produto() e desativar_produto(). Validar código, nome, preço, estoque mínimo e categoria.

```
git add src/produtos.py
git commit -m "feat: implementa cadastro e gerenciamento de produtos"
```

Commit 3 — Filtros

- Arquivos: src/produtos.py
- Adicionar busca por nome, código e categoria.

```
git add src/produtos.py
git commit -m "feat: adiciona filtros de consulta de produtos"
```

Commit 4 — Testes

- Arquivos: tests/test_produtos.py e tests/test_categorias.py
- Criar testes das regras principais de produtos e categorias.

```
git add tests/test_produtos.py tests/test_categorias.py
git commit -m "test: adiciona testes de produtos e categorias"
```

Envio da branch

```
git push -u origin feature/produtos
```

PESSOA 2 — Entradas, Saídas e Estoque

Branch:

```
git checkout develop
git pull origin develop
git checkout -b feature/movimentacoes
```

Commit 1 — Entrada de estoque

- Arquivos: src/movimentacoes.py
- Implementar registrar_entrada() e atualizar o saldo do produto.

```
git add src/movimentacoes.py
git commit -m "feat: implementa entrada de produtos no estoque"
```

Commit 2 — Saída de estoque

- Arquivos: src/movimentacoes.py
- Implementar registrar_saida() e atualizar o saldo.

```
git add src/movimentacoes.py
git commit -m "feat: implementa saida de produtos do estoque"
```

Commit 3 — Validação de saldo

- Arquivos: src/movimentacoes.py
- Impedir saída acima do estoque disponível.

```
git add src/movimentacoes.py
git commit -m "feat: impede saida acima do estoque disponivel"
```

Commit 4 — Histórico

- Arquivos: src/movimentacoes.py
- Adicionar listar_movimentacoes(), listar_entradas() e listar_saidas().

```
git add src/movimentacoes.py
git commit -m "feat: adiciona historico de movimentacoes"
```

Commit 5 — Testes

- Arquivos: tests/test_movimentacoes.py
- Testar entradas, saídas, atualização de saldo e bloqueio de saldo negativo.

```
git add tests/test_movimentacoes.py
git commit -m "test: adiciona testes de entrada saida e saldo"
```

Envio da branch

```
git push -u origin feature/movimentacoes
```

PESSOA 3 — Dashboard e Relatórios

Branch:

```
git checkout develop
git pull origin develop
git checkout -b feature/relatorios
```

Commit 1 — Estoque atual

- Arquivos: src/relatorios.py
- Implementar relatorio_estoque().

```
git add src/relatorios.py
git commit -m "feat: implementa relatorio de estoque atual"
```

Commit 2 — Estoque baixo e zerado

- Arquivos: src/relatorios.py
- Implementar produtos_estoque_baixo() e produtos_sem_estoque().

```
git add src/relatorios.py
git commit -m "feat: adiciona relatorio de estoque baixo e zerado"
```

Commit 3 — Movimentações

- Arquivos: src/relatorios.py
- Adicionar movimentacoes_por_periodo() e movimentacoes_por_produto().

```
git add src/relatorios.py
git commit -m "feat: adiciona relatorios de movimentacoes"
```

Commit 4 — Indicadores gerais

- Arquivos: src/relatorios.py
- Apresentar totais de produtos, produtos abaixo do mínimo, entradas e saídas.

```
git add src/relatorios.py
git commit -m "feat: adiciona indicadores gerais do estoque"
```

Commit 5 — Testes

- Arquivos: tests/test_relatorios.py
- Validar que os totais dos relatórios correspondem às movimentações cadastradas.

```
git add tests/test_relatorios.py
git commit -m "test: adiciona testes dos relatorios"
```

Envio da branch

```
git push -u origin feature/relatorios
```

PESSOA 4 — Funcionários, Login e Permissões

Branch:

```
git checkout develop
git pull origin develop
git checkout -b feature/autenticacao
```

Commit 1 — Funcionários

- Arquivos: src/funcionarios.py
- Implementar cadastrar_funcionario(), listar_funcionarios(), editar_funcionario() e desativar_funcionario().

```
git add src/funcionarios.py
git commit -m "feat: implementa gerenciamento de funcionarios"
```

Commit 2 — Autenticação

- Arquivos: src/autenticacao.py
- Implementar login() e validar_usuario().

```
git add src/autenticacao.py
git commit -m "feat: implementa autenticacao de usuarios"
```

Commit 3 — Hash de senha

- Arquivos: src/autenticacao.py + requirements.txt
- Adicionar bcrypt e armazenar somente hash de senha.

```
git add src/autenticacao.py requirements.txt
git commit -m "feat: adiciona hash seguro de senhas com bcrypt"
```

Commit 4 — Permissões

- Arquivos: src/autenticacao.py e src/funcionarios.py
- Implementar perfis, por exemplo ADMIN, ESTOQUISTA e CONSULTA.

```
git add src/autenticacao.py src/funcionarios.py
git commit -m "feat: implementa controle de permissoes"
```

Commit 5 — Testes

- Arquivos: tests/test_funcionarios.py e tests/test_autenticacao.py
- Testar usuários, login, senha e restrições de permissão.

```
git add tests/test_funcionarios.py tests/test_autenticacao.py
git commit -m "test: adiciona testes de usuarios e autenticacao"
```

Envio da branch

```
git push -u origin feature/autenticacao
```

PESSOA 5 — Banco, Fornecedores, Auditoria e Integração

Branch:

```
git checkout develop
git pull origin develop
git checkout -b feature/fornecedores-integracao
```

Commit 1 — Banco de dados

- Arquivos: src/banco.py
- Implementar conectar(), criar_tabelas() e fechar_conexao(). Esta entrega deve ocorrer cedo para servir de base aos outros módulos.

```
git add src/banco.py
git commit -m "feat: implementa conexao e estrutura inicial do banco"
```

Commit 2 — Fornecedores

- Arquivos: src/fornecedores.py
- Implementar cadastrar_fornecedor(), listar_fornecedores(), buscar_fornecedor(), editar_fornecedor() e desativar_fornecedor().

```
git add src/fornecedores.py
git commit -m "feat: implementa gerenciamento de fornecedores"
```

Commit 3 — Auditoria

- Arquivos: src/auditoria.py
- Registrar eventos relevantes, como login, cadastro, entrada, saída e alteração de produto.

```
git add src/auditoria.py
git commit -m "feat: implementa registro de auditoria"
```

Commit 4 — Integração principal

- Arquivos: src/main.py
- Integrar os módulos e montar o fluxo principal do sistema.

```
git add src/main.py
git commit -m "feat: integra modulos no fluxo principal do sistema"
```

Commit 5 — Testes

- Arquivos: tests/test_fornecedores.py
- Criar testes de fornecedores e apoiar os testes de integração.

```
git add tests/test_fornecedores.py
git commit -m "test: adiciona testes de fornecedores"
```

Envio da branch

```
git push -u origin feature/fornecedores-integracao
```

3. Histórico esperado de commits

PESSOA 1

- feat: implementa gerenciamento de categorias
- feat: implementa cadastro e gerenciamento de produtos
- feat: adiciona filtros de consulta de produtos
- test: adiciona testes de produtos e categorias

PESSOA 2

- feat: implementa entrada de produtos no estoque
- feat: implementa saida de produtos do estoque
- feat: impede saida acima do estoque disponivel
- feat: adiciona historico de movimentacoes
- test: adiciona testes de entrada saida e saldo

PESSOA 3

- feat: implementa relatorio de estoque atual
- feat: adiciona relatorio de estoque baixo e zerado
- feat: adiciona relatorios de movimentacoes
- feat: adiciona indicadores gerais do estoque
- test: adiciona testes dos relatorios

PESSOA 4

- feat: implementa gerenciamento de funcionarios
- feat: implementa autenticacao de usuarios
- feat: adiciona hash seguro de senhas com bcrypt
- feat: implementa controle de permissoes
- test: adiciona testes de usuarios e autenticacao

PESSOA 5

- feat: implementa conexao e estrutura inicial do banco
- feat: implementa gerenciamento de fornecedores
- feat: implementa registro de auditoria
- feat: integra modulos no fluxo principal do sistema
- test: adiciona testes de fornecedores

4. Regras para a avaliação pelo Git

- Não concentrar toda a atividade em um único commit.
- Cada commit deve representar uma funcionalidade, correção ou conjunto de testes específico.
- Evitar mensagens genéricas como "minha parte", "alterações" ou "final".
- Preferir git add com os arquivos relacionados ao commit, em vez de usar git add . automaticamente.
- Cada integrante deve fazer commits usando sua própria conta/configuração do Git.
- Cada integrante deve fazer push da própria branch e abrir Pull Request para develop.
- Antes de atualizar main, a equipe deve testar a versão integrada em develop.

Sequência recomendada: Pessoa 5 prepara primeiro a base do banco -> integra em develop -> todos atualizam develop -> cada pessoa continua sua feature branch. Isso reduz conflitos e incompatibilidades.

5. Fluxo diário de desenvolvimento

```
git checkout feature/SUA-BRANCH
git pull

# editar e testar os arquivos

git status
git add ARQUIVOS_DO_COMMIT
git commit -m "feat: descricao objetiva"
git push
```

6. Critério de entrega individual

- Pessoa 1: cadastrar, editar, filtrar e desativar produtos e administrar categorias.
- Pessoa 2: registrar entrada e saída, alterar saldo e bloquear saída sem estoque suficiente.
- Pessoa 3: gerar relatórios coerentes com as movimentações cadastradas.
- Pessoa 4: realizar login, cadastrar funcionário e demonstrar restrição de permissão.
- Pessoa 5: cadastrar fornecedor, demonstrar persistência, auditoria e integração entre módulos.
- Todos: entregar código versionado, testes, evidências de execução e explicação das decisões técnicas.